

CSA0613

Design and Analysis of Algorithms

CO1_ASSESSMENT TOOL 1_Application-Based Question

Divyesh S P (192424147)

Question 1: Application: Real-Time Search Index Compression

A search indexing system reduces its input size by half at each recursive step. At every level, it performs additional indexing work proportional to the logarithm of the current input size. Apply the substitution method to solve the recurrence $T(n) = T(n/2) + \log_{10} n$. Expand the recurrence step-by-step and derive the final asymptotic time complexity. Analyze how the recursion depth influences performance. Interpret the efficiency using asymptotic notation.

Question 2: Application: Cloud Data Backup System

A cloud backup system divides data into smaller chunks recursively: $T(n) = 2T(n/2) + n \log n$. Apply the Master Theorem and determine the time complexity. Identify the correct case and justify your answer. Discuss how the algorithm scales with increasing data size and interpret using asymptotic notation.

Real-Time Search Index Compression

Given Recurrence

$$T(n) = T(n/2) + \log n \quad T(n) = T(n/2) + \log n \quad T(n) = T(n/2) + \log n$$

Substitution Method (Expansion)

Expand the recurrence:

$$\begin{aligned} T(n) &= T(n/2) + \log n \\ T(n) &= T(n/2) + \log n \\ T(n) &= T(n/2) + \log n \\ &= T(n/4) + \log(n/2) + \log n \\ &= T(n/4) + \log(n/2) + \log n \\ &= T(n/4) + \log(n/2) + \log n \\ &= T(n/8) + \log(n/4) + \log(n/2) + \log n \\ &= T(n/8) + \log(n/4) + \log(n/2) + \log n \\ &= T(n/8) + \log(n/4) + \log(n/2) + \log n \end{aligned}$$

After k expansions,

$$\begin{aligned} T(n) &= T(n/2^k) + \sum_{i=0}^{k-1} \log(n/2^i) \\ T(n) &= T(n/2^k) + \sum_{i=0}^{k-1} (\log n - \log 2^i) \\ T(n) &= T(n/2^k) + \sum_{i=0}^{k-1} (\log n - i) \end{aligned}$$

The recursion stops when

$$n/2^k = 1 \quad \frac{n}{2^k} = 1 \quad 2^k = n$$

Therefore,

$$k = \log_2 n \quad 2^k = n \quad k = \log_2 n$$

Each recursive level contributes $O(\log n)$ work, and there are $O(\log n)$ levels.

Hence,

$$\begin{aligned} T(n) &= O(\log n \times \log n) \\ T(n) &= O(\log n \times \log n) \\ T(n) &= O(\log n \times \log n) \\ T(n) &= O((\log n)^2) \end{aligned}$$

Recursion Depth Analysis

- Input size becomes half at every recursive call.
 - Number of recursive levels = $\log_2 n$.
 - Work at each level is proportional to $\log n$.
 - Total work = $\log n \times \log n$.
-

Asymptotic Time Complexity

$O((\log n)^2)$ $O((\log n)^2)$ $O((\log n)^2)$

Source Code

```
import math

def search_index_compression(n):
    if n <= 1:
        return 0
    return math.log2(n) + search_index_compression(n // 2)

n = int(input("Enter the input size: "))

result = search_index_compression(n)

print("Total indexing work:", round(result, 2))
print("Asymptotic Time Complexity: O((log n)^2)")
```

Output

```
▶ #Question 1 Application: Real-Time Search Index Compression
import math

def search_index_compression(n):
    if n <= 1:
        return 0
    return math.log2(n) + search_index_compression(n // 2)

n = int(input("Enter the input size: "))

result = search_index_compression(n)

print("Total indexing work:", round(result, 2))
print("Asymptotic Time Complexity: O((log n)^2)")

... Enter the input size: 16
Total indexing work: 10.0
Asymptotic Time Complexity: O((log n)^2)
```

Application: Cloud Data Backup System

Given Recurrence

$$T(n) = 2T(n/2) + n \log n \quad T(n) = 2T(n/2) + n \log n \quad T(n) = 2T(n/2) + n \log n$$

Applying the Master Theorem

The recurrence is of the form:

$$T(n) = aT(n/b) + f(n) \quad T(n) = aT(n/b) + f(n) \quad T(n) = aT(n/b) + f(n)$$

where,

- $a = 2$
- $b = 2$
- $f(n) = n \log n$

Calculate:

$$n \log_b a = n \log_2 2 = n \cdot 1 = n \quad n^{\log_b a} = n^1 = n \quad n \log_b a = n \log_2 2 = n$$

Compare:

- $f(n) = n \log n$
- $n \log_b a = n$

Since

$$f(n) = \Theta(n \log n) = \Theta(n \log_b a \log n) \quad f(n) = \Theta(n \log n) = \Theta(n \log_b a \log n) \\ = \Theta(n^{\log_b a} \log n) \quad f(n) = \Theta(n \log n) = \Theta(n \log_b a \log n)$$

this matches **Master Theorem Case 2**, where

$$f(n) = \Theta(n \log_b a \log^k n), k=1 \quad f(n) = \Theta(n^{\log_b a} \log^k n), \quad k=1 \\ f(n) = \Theta(n \log_b a \log^k n), k=1$$

Therefore,

$$T(n) = \Theta(n \log^{k+1} n) \quad T(n) = \Theta(n \log^{k+1} n) \quad T(n) = \Theta(n \log^{k+1} n) \\ T(n) = \Theta(n (\log n)^2) \quad T(n) = \Theta(n (\log n)^2) \quad T(n) = \Theta(n (\log n)^2)$$

Case Identification

Master Theorem: Case 2

Because

$$f(n) = \Theta(n \log^2 n) = \Theta(n \log^2 n)$$
$$f(n) = \Theta(n \log^2 n) = \Theta(n \log^2 n)$$

Scalability Analysis

- The data is divided into **two equal halves** recursively.
- Each level performs **$n \log n$** work.
- There are **$\log n$** levels of recursion.
- Hence, the total running time grows as:

$$n \log n \times \log n = n (\log n)^2$$

As the data size increases, the algorithm scales efficiently but is slightly slower than $O(n \log n)$ because of the extra logarithmic factor.

Asymptotic Time Complexity

$$\Theta(n (\log n)^2)$$

Source Code

```
import math

def cloud_backup(n):
    if n <= 1:
        return 1
    return 2 * cloud_backup(n // 2) + n * math.log2(n)

n = int(input("Enter the data size: "))

result = cloud_backup(n)

print("Estimated backup work:", round(result, 2))
print("Time Complexity:  $\Theta(n (\log n)^2)$ ")
```

Output

```
import math

def cloud_backup(n):
    if n <= 1:
        return 1
    return 2 * cloud_backup(n // 2) + n * math.log2(n)

n = int(input("Enter the data size: "))

result = cloud_backup(n)

print("Estimated backup work:", round(result, 2))
print("Time Complexity:  $\Theta(n (\log n)^2)$ ")
```

```
Enter the data size: 8
Estimated backup work: 56.0
Time Complexity:  $\Theta(n (\log n)^2)$ 
```